

Beating Round 2

In class, someone asked about how beating works when there are multiple signals as opposed to just two. I said that in general, there isn't any special pattern, and you will just get kind of random loudening/softening. That's indeed generally true, but if you pick your beating signals very, very carefully, then you can get dramatically different behavior. We'll look a little bit at this.

1. Two-signal Beating

If we have two closely-spaced in frequency sine/cosine waves, we saw that we get *beating*. Instead of hearing two slightly separate notes, we hear one note that gets louder and softer. Mathematically, that's because

$$\cos(2\pi(\nu_0 + d\nu)t) + \cos(2\pi(\nu_0 - d\nu)t) = 2 \cos(2\pi\nu_0 t) \cos(2\pi d\nu t) \quad (1)$$

We have a fast sine wave $2 \cos(2\pi\nu_0 t)$ that our ear hears as a pitch that gets modulated by $\cos(2\pi d\nu t)$. If $d\nu$ is small enough (less than about 20 Hz) then our ears hear this as the main pitch at ν_0 getting louder and softer as the $d\nu$ cosine goes between 1 and 0 (and -1 and back to 0).

What we have written down assumes that our two notes start at exactly the same time. Generally this won't be true, and one note will have started slightly later than the other, so what we *really* hear is

$$\cos(2\pi(\nu_0 + d\nu)(t - dt)) + \cos(2\pi(\nu_0 - d\nu)t) \quad (2)$$

Equation 2 looks a lot like Equation 1, except for the dt . That dt says that the first note started just a little bit before the second note. If you go through the (now much more painful!) math, you'll discover that nothing really changes. *When* the beats happen will change, but the beating pattern itself remains the same.

2. Four-note Beating

Now let's look at what happens when we have four notes going at the same time. We'll start with the notes starting together ("in phase" is the usual technical term), with frequencies $\nu_0 - 2d\nu, \nu_0 - d\nu, \nu_0 + d\nu, \nu_0 + 2d\nu$. We can sum the $\pm d\nu$ pair and the $\pm 2d\nu$ pair to get:

$$2 \cos(2\pi\nu_0 t) \cos(2\pi d\nu t) + 2 \cos(2\pi\nu_0 t) \cos(2\pi 2d\nu t) = 2 \cos(2\pi\nu_0 t) (\cos(2\pi d\nu t) + \cos(4\pi d\nu t)) \quad (3)$$

We still get a fast sine wave times an envelope, but this time the envelope is the sum of *two* cosine waves. One of them goes twice as fast as the other. The ensuing envelope pattern is in Figure ???. When $t = 0$, the two cosines are both +1, and so they sum to +2. That's the loudest our signal could ever possibly get since all the signals are aligned. You'll notice that where before we would have expected an equally loud beat when the envelope goes negative, now we don't get that because we never manage to line all of our waves up with -1 at the same time. This beating

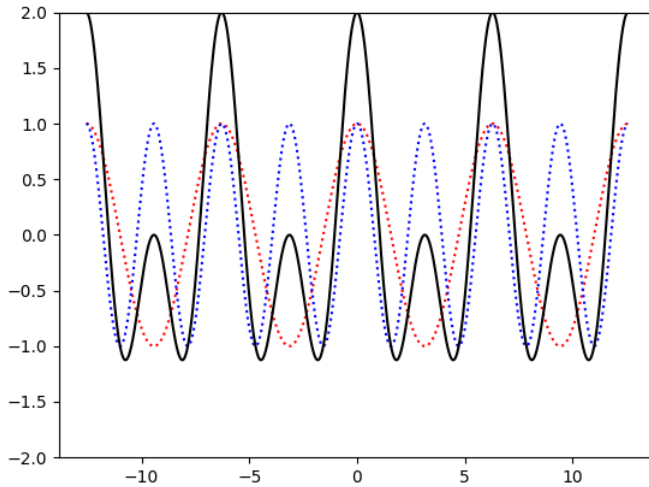


Fig. 1.— Sum of two cosines (black). The first (dotted blue) has exactly twice the frequency of the second (dotted red). The sum has a peak at 2 where the two waves are in-phase, but they are never in-phase to give a negative peak at -2. If this were an envelope, the beats would be louder where the sum peaks at +2.

pattern is going to have the occasional loud beat (where the envelope goes to 2) separated by some lower-level warbling.

With the 4-note beating, we can also see that now the phases *do* matter. I could shift the fast signal by half of a wave, and now the signals will combine at -2 instead of 2. That outcome is shown in Figure ??, and our ear will hear the same qualitative pattern as before. Because our ear hears *changes* in pressure, flipping the sign of the envelope does not change what we hear. We now have the freedom to do something a little different, though. We could shift the $2d\nu$ pair by a quarter of a wavelength in time. This gives us a beat pattern shown in Figure ?. Now the qualitative behavior *is* different. We no longer ever get all four waves in phase, so the beat pattern will change. Instead of loud beats alternating with lower-level warbling, we get less-loud beats twice as often. For typical frequencies we hear, the time shift is on the order of a thousandth of a second, so getting four real-life players to produce these exact patterns would be *extremely* hard. We can easily do it with computers though.

3. Many Signal Beating

If we have a bunch of signals at closely-spaced frequencies, we can now start to get an idea of what to expect. If we start them all at random times, we'll still get some sort of beating, but it won't usually be any particular pattern. Just some low-level warbling about. You can listen to `beats_1_notes_random.wav` on github to hear what this sounds like. However, if we start all our signals at *exactly* the same time, they all start out in-phase, so we get a very loud beat. Then we get

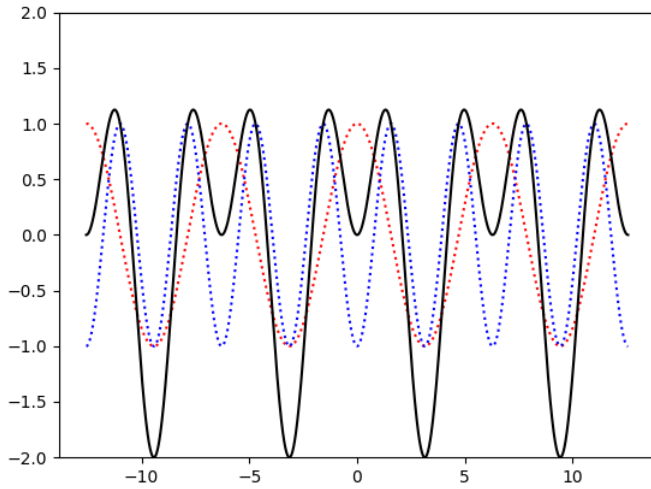


Fig. 2.— Sum of two cosines (black). The first (dotted blue) has exactly twice the frequency of the second (dotted red). This time the blue (high-frequency) wave has been shifted by half a wavelength. The envelope now peaks at -2. We would hear the same overall pattern as in Figure ??, though.

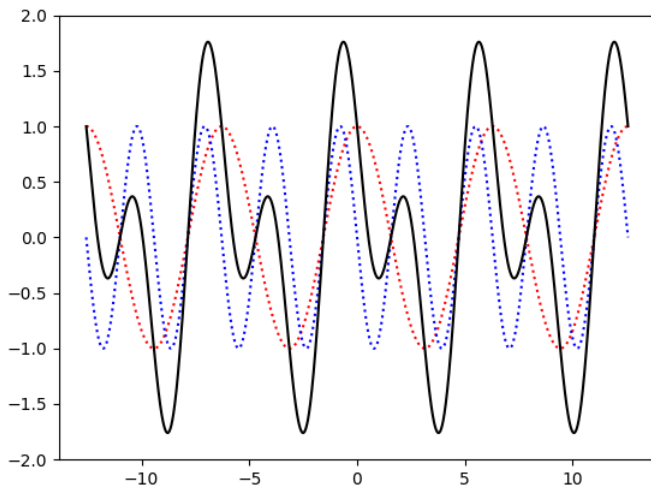


Fig. 3.— Sum of two cosines (black). The first (dotted blue) has exactly twice the frequency of the second (dotted red). Now the blue (high-frequency) wave has been shifted by a quarter of a wavelength. Now we never get the signals perfectly aligned, and so we hear beats of equal loudness but twice as fast as the other examples. Unlike the 2-wave pattern, the relative timing matters!

low-level warbling until they all come back in phase, and we get another loud beat. Roughly speaking, the spacing in frequency tells us how long it takes for a cycle to repeat, and the number of different waves tells us how sharp in time the beat will be. If you listen to `beats_1_notes_nophase.wav`,

you'll hear this beating pattern with 16 separate sinewaves. The beats repeat every few seconds with one loud beat, with low-level warbling between the beats. If you look at [this python code](#), you can play with the variable `nnu` to hear this in action. The larger you set `nnu` (which is the total number of sine waves to use), the shorter the beat will be, and the louder it will be relative to the background.

This is sometimes classified as an audio illusion. Mathematically, we have a bunch of sine waves with equal amplitude, but our ear doesn't hear this at all. Our ear hears a single note with a lot of amplitude modulation going on. We can go even further, with a bunch of different notes played together at the same time. If you listen to `beats_8_notes_random.wav`, you'll hear the notes of the major scale played simultaneously. Each one has 16 separate closely-spaced sinewaves going into it. They are all started at random times relative to each other, and the effect is a sort of shimmering sound-wall. Not at all unpleasant (at least to my ear), unlike the beating from a single pair of tones. If you're very careful with the timing, you can have the notes be in-phase, and you can even have the different notes have their envelopes peak at different times. You can hear this in `beats_8_notes_nophase.wav`, where now I've carefully aligned the timing of each wave to trick your ear into hearing a major scale. Again, the actual signals are a set of sine waves with the same amplitude, but your ear now hears notes in a scale. We can even play some simple melodies - check out `beats_16_notes_nophase.wav` for a snippet.

4. Discussion

Although the beat pattern from two closely-spaced sine waves is rather simple, with a single note modulated by an envelope that gets louder and softer in time. Adding in more notes quickly makes the envelopes more complicated. If we pick the start times (or phases) randomly, the envelopes tend to just kind of wander around. However, if we're very carefully with how we pick the phases, we can produce what our ear starts to hear as individual notes, and we can even pick when the notes are played, and how long they last. This opens up a wide range of sounds we can produce, again with just the sum of uniform sine waves. I haven't shown you *how* I picked the timings for the more complicated cases, but the 4-note beat pattern does start to show how you might go about picking your phases. Later on, we'll at least touch on Fourier transforms, which will let us turn *any* sound into a sum of sines/cosines. By any sound, I do mean *any* sound, including everything from Gregorian chants to symphonies to Angine de Poitrine!